LAMP-KV: Memory-Pressure-Aware Mixed-Precision KV-Cache Compression for Memory-Constrained LLM Inference

John Cheung

College of Professional and Continued Education

May 2026

Abstract

Long-context large language model (LLM) inference is increasingly constrained by key-value (KV) cache memory. Prior work has already established several important directions, including paged KV-cache management, low-bit KV quantization, quality-adaptive KV quantization, and token selection or eviction. Therefore, a new academic contribution should not simply repeat fixed INT4 or INT2 KV-cache quantization. This paper proposes *LAMP-KV*, a memory-pressure-aware mixed-precision policy that selects KV-cache precision at block granularity. The core idea is to treat the quantizer as a set of available actions and to study the runtime policy problem: when should a system use INT8, INT4, INT3, INT2, or FP16 fallback under changing memory pressure? A reproducible Python artifact for Windows and Linux is available at https://github.com/johncheungmk/LAMP-KV. A pilot test on a Lenovo Yoga Slim 7 14Q8X9 Windows ARM notebook with LPDDR5X memory shows that fixed INT8, INT4, INT3, and INT2 achieve 1.95×, 3.82×, 5.02×, and 7.31× compression, respectively, on a synthetic KV-cache tensor. At medium pressure, LAMP-KV selects a mixed INT4/INT8 policy, achieving 3.01× compression while reducing mean absolute error from 0.1043 under fixed INT4 to 0.0738. These results support the feasibility of pressure-aware bit allocation as a research direction, while real-model KV traces and end-to-end LLM quality evaluation remain required before deployment claims.

Keywords: large language model inference; KV cache; quantization; mixed precision; memory pressure; long context; on-device inference.

Artifact availability: The supplementary testing program and README are available at https://github.com/johncheungmk/LAMP-KV.

1 Introduction

Autoregressive LLM inference stores key and value activations from previous tokens so that each subsequent decoding step can attend over the existing context without recomputing the entire prefix. This KV cache is essential for efficient generation, but its memory footprint grows linearly with sequence length, batch size, number of layers, number of heads, and head dimension. For long-context and batched inference, KV-cache memory can become a practical bottleneck even when model weights fit in memory.

Strong baselines already exist. PagedAttention and vLLM reduce KV-cache waste through block-level memory management and sharing [1]. KIVI proposes tuning-free asymmetric 2-bit KV quantization with per-channel key quantization and per-token value quantization [2]. KVQuant studies sub-4-bit KV-cache quantization with techniques such as pre-RoPE key quantization, non-uniform datatypes, and outlier handling [3]. QAQ proposes quality-adaptive KV-cache quantization with outlier-aware and attention-aware handling [4]. H_2O and SnapKV reduce KV storage by retaining or selecting important tokens rather than storing all positions at full cost

[5, 6]. Recent work has also begun to investigate adaptive bit allocation for on-device LLMs [7]. These results mean that plain KV-cache quantization is not a sufficient novelty claim.

This paper therefore narrows the research question. Rather than claiming to invent KV quantization, we ask whether a simple, reproducible, device-aware policy can provide useful intermediate operating points between fixed-bit baselines. The proposed policy, LAMP-KV, receives a memory-pressure signal and chooses precision per KV block. At low memory pressure, it should behave conservatively. At high pressure, it should become more aggressive. At medium pressure, it should mix precisions and preserve high precision for blocks that appear more sensitive to quantization.

1.1 Motivation from an Earlier Negative Result

The work began with a DASH/ASP-QADD-style block-local delta-coding prototype for static model blocks. A Windows notebook run on a 4096×4096 synthetic matrix with block size 128 and 3-bit residuals reduced storage from 33,554,432 bytes to 7,340,032 bytes, corresponding to 4.57× compression relative to FP16 and 1.21× relative to an INT4-like baseline. However, the pure Python/NumPy reference required approximately 21.2 seconds for encoding and 21.1 seconds for decoding. This demonstrated compression potential but weak practical inference value. LAMP-KV is motivated by this result: byte savings alone are insufficient unless the policy targets an actual runtime memory bottleneck and can fall back to simple, established representations.

1.2 Contributions

This paper makes the following contributions:

1. It reformulates the research objective away from custom static weight-block delta coding and toward adaptive KV-cache precision selection.
2. It proposes LAMP-KV, a memory-pressure-aware mixed-precision policy that chooses among INT2, INT3, INT4, INT8, and FP16 fallback at KV-block granularity.
3. It provides a reproducible Python artifact for peer review at `https://github.com/johncheungmk/LAMP-KV`, supporting synthetic KV tensors and optional real KV tensors in NPZ format.
4. It reports pilot Windows/LPDDR5X notebook results showing that the adaptive policy provides a middle point between fixed INT8 and fixed INT4: 3.01× compression at medium pressure with lower reconstruction error than fixed INT4.

2 Background and Related Work

2.1 KV-Cache Memory Growth

For a decoder-only transformer, approximate FP16 KV-cache memory is

$$M_{KV} = 2 \cdot B \cdot S \cdot L \cdot H \cdot D \cdot 2 \quad \text{bytes}, \tag{1}$$

where B is batch size, S is sequence length, L is number of layers, H is number of heads, D is head dimension, the leading factor of 2 accounts for keys and values, and the final factor of 2 is the byte width of FP16. This expression explains why long-context and batched inference rapidly increase memory consumption.

2.2 Paged KV Management

vLLM's PagedAttention uses a paging-inspired KV-cache abstraction and supports flexible sharing across requests [1]. This work is complementary to LAMP-KV. PagedAttention improves allocation and reduces fragmentation, while LAMP-KV studies how many bits should be used inside retained KV blocks.

2.3 KV Quantization

KIVI observes that key and value caches have different distributional behavior and proposes asymmetric 2-bit quantization: keys are quantized per channel and values per token [2]. KVQuant further targets sub-4-bit KV quantization with outlier-aware and non-uniform representations [3]. QAQ proposes quality-adaptive KV quantization and highlights that key and value caches can require different treatment [4]. LAMP-KV adopts the KIVI-style axis convention in its reference simulator but treats it as a baseline design choice, not as a novelty claim.

2.4 KV Eviction and Token Selection

H_2O observes that a subset of heavy-hitter tokens contributes disproportionately to attention and designs a KV eviction policy around this observation [5]. SnapKV selects clustered important positions using an observation window near the end of the prompt [6]. These methods reduce the number of stored tokens. LAMP-KV instead changes precision for stored blocks. The two families are potentially composable.

2.5 Adaptive Precision

Recent adaptive KV-cache work confirms that fixed global precision may waste bits on low-impact states while over-compressing important states [7]. This makes the novelty boundary important. LAMP-KV does not claim that adaptive precision is untouched. Its narrower contribution is a memory-pressure-aware, block-level, reproducible policy artifact intended for early-stage evaluation on commodity memory-constrained systems.

3 LAMP-KV Policy

3.1 Block Representation

The simulator flattens key and value tensors into matrices of shape $[T, D]$, where $T = B \cdot S \cdot L \cdot H$ and D is head dimension. The token dimension is partitioned into blocks of N rows. Key blocks use per-channel scaling, and value blocks use per-token scaling. This follows the convention used by KIVI-style quantization but allows LAMP-KV to focus on policy selection.

For bit width b, symmetric quantization is defined as

$$q = \text{clip}\left(\text{round}(x/s), -2^{b-1}, 2^{b-1} - 1\right), \tag{2}$$

with dequantized reconstruction

$$\hat{x} = q \cdot s. \tag{3}$$

The estimated storage includes packed quantized values and FP16 scale overhead. Scale overhead is reported explicitly because small blocks can lose much of their compression benefit to metadata.

3.2 Memory-Pressure Signal

The policy receives a scalar pressure value $p \in [0, 1]$. A value of 0 means memory is comfortable, and a value of 1 means memory pressure is high. In a full runtime, this signal could come from available memory, KV-cache occupancy, request queue pressure, or operating-system memory feedback. In the artifact, it is supplied manually to make the sweep reproducible.

3.3 Block Importance Proxy

Because the simulator does not run a real attention kernel, it estimates block sensitivity with a simple reproducible proxy:

$$I(x) = \text{mean}(x^2) + 0.1 \cdot P_{99}(|x|). \tag{4}$$

This proxy favors blocks with high energy or heavy tails. A full implementation should replace it with a better estimator, such as attention mass, layer sensitivity, calibration loss, or output-token divergence.

3.4 Threshold Adjustment

Given base thresholds τ_{mean} and τ_{max}, LAMP-KV computes pressure-adjusted and importance-adjusted thresholds. Higher pressure relaxes thresholds, while higher estimated importance tightens them. In the reference implementation, the pressure factor is

$$f_p = 0.75 + 1.25p. \tag{5}$$

The importance factor is clipped to avoid extreme behavior. Each block is then tested against mean absolute error and maximum absolute error thresholds.

3.5 Precision Selection

For each block, the policy tries INT2, INT3, INT4, and INT8 in ascending storage cost order. It selects the first precision satisfying both mean and maximum error constraints. If none satisfies the constraints, it falls back to FP16. The policy is intentionally simple: it is meant to be auditable and reproducible before being replaced by a learned or calibration-based controller.

4 Experimental Setup

4.1 Artifact

The artifact contains:

- `paper.tex`: LaTeX source;
- `references.bib`: bibliography;
- `scripts/lamp_kv_policy_test.py`: main simulator;
- `scripts/run_matrix_windows.bat`: Windows experiment runner;
- `scripts/summarize_results.py`: JSON-to-CSV summary helper;
- `README.md`: usage instructions for GitHub peer review.

4.2 Pilot Platform

The pilot results were produced on a Windows 11 notebook with Python 3.13.5, NumPy 2.4.4, and a Qualcomm ARMv8 64-bit processor. The platform is a Lenovo Yoga Slim 7 14Q8X9-class notebook using LPDDR5X memory. These results should be interpreted as notebook portability and memory/error policy results, not as server DDR5 or GPU serving measurements.

4.3 Synthetic KV Tensor

The main pressure sweep uses a synthetic KV tensor with

$$B = 1, \quad S = 1024, \quad L = 32, \quad H = 32, \quad D = 128. \tag{6}$$

The resulting number of entries per cache is 134,217,728, and the combined FP16 key/value cache is 536,870,912 bytes. The synthetic distribution is Gaussian with sparse outliers controlled by an outlier probability of 0.001 and outlier scale 6.0.

4.4 Baselines and Metrics

The baselines are fixed INT8, INT4, INT3, and INT2, using the same scale-axis convention as the adaptive policy. Metrics are compression ratio relative to FP16, estimated bytes including scale metadata, mean absolute error, 95th-percentile block MAE, mean 95th-percentile element error, maximum error, and bit-allocation histogram.

5 Pilot Results

5.1 Fixed-Precision Baselines

Table 1 reports the fixed-bit results for sequence length 1024. Lower precision improves compression but increases reconstruction error.

Table 1: Fixed-precision results for synthetic KV cache, $S = 1024$. Bytes include scale metadata.

Method	Bytes	Compression	Mean MAE	Max error
INT8	274,726,912	1.95×	0.00575	0.111
INT4	140,509,184	3.82×	0.10434	2.008
INT3	106,954,752	5.02×	0.24029	4.230
INT2	73,400,320	7.31×	0.63448	10.148

5.2 Pressure Sweep

Table 2 shows the adaptive policy under pressure values 0.0, 0.5, and 1.0. The policy behaves as intended. At pressure 0.0, it is conservative and selects only INT8. At pressure 0.5, it mixes INT4 and INT8. At pressure 1.0, it nearly converges to fixed INT4.

Table 2: LAMP-KV adaptive policy results, $S = 1024$.

Pressure	Bytes	Compression	Mean MAE	INT4 blocks	INT8 blocks
0.0	274,726,912	1.95×	0.00575	0	32,768
0.5	178,085,888	3.01×	0.07376	23,594	9,174
1.0	140,517,376	3.82×	0.10433	32,766	2

The most useful operating point is pressure 0.5. Compared with fixed INT8, it improves compression from 1.95× to 3.01×. Compared with fixed INT4, it reduces mean absolute error from 0.10434 to 0.07376 and lowers maximum error from 2.008 to 1.365, at the cost of lower compression than fixed INT4. This supports the policy claim: adaptive mixed precision can expose intermediate memory/error tradeoffs that fixed global precision cannot.

5.3 Sequence-Length 512 Smoke Test

A lighter smoke test with sequence length 512 produced the same qualitative behavior. At pressure 0.5, the adaptive policy selected 11,785 INT4 blocks and 4,599 INT8 blocks, achieved 3.01× compression, and reported mean absolute error 0.07368. This is useful for reviewer machines with limited memory.

5.4 Interpretation

The result should be interpreted carefully. It shows that LAMP-KV can implement a memory-pressure-controlled precision transition from INT8-like behavior to INT4-like behavior. It does not yet prove real model quality preservation, token-level agreement, or serving latency improvement. These require real KV tensors and integration into an LLM runtime.

6 Reproducibility Instructions

On Windows, install NumPy:

```
python -m pip install numpy
```

Run the light test:

```
python scripts/lamp_kv_policy_test.py --batch 1 --seq-len 512 \
  --layers 32 --heads 32 --head-dim 128 --block-tokens 64 \
  --pressure 0.5 --seed 7
```

Run the pressure sweep:

```
python scripts/lamp_kv_policy_test.py --pressure 0.0 --seq-len 1024 --seed 7
python scripts/lamp_kv_policy_test.py --pressure 0.5 --seq-len 1024 --seed 7
python scripts/lamp_kv_policy_test.py --pressure 1.0 --seq-len 1024 --seed 7
```

Use `-out results/name.json` to save individual results. Use `scripts/summarize_results.py` to convert saved JSON files to CSV. The README in the artifact contains exact Windows commands and peer-review instructions. The current project repository is https://github.com/johncheungmk/LAMP-KV.

7 Limitations and Threats to Validity

Synthetic data. The pilot results use synthetic KV tensors. Real KV tensors may have different distributions, outlier behavior, layer sensitivity, and attention sensitivity.

Proxy metrics. Mean absolute error and maximum error are not language-quality metrics. They are early-stage filters. A full paper revision should include perplexity, token agreement, and downstream task accuracy.

Python performance. The simulator is a reference implementation. Its wall-clock time should not be used as serving latency.

Overlap with prior work. KV-cache quantization, quality-adaptive quantization, and adaptive bit allocation are active areas. The contribution here is a constrained, pressure-aware policy artifact and pilot evaluation, not a claim that adaptive KV compression is entirely new.

Hardware. The notebook platform uses LPDDR5X memory and an ARM processor. It is relevant for on-device and memory-constrained testing, but does not validate DDR5 server behavior or GPU kernel performance.

8 Future Work

The next step is to capture real KV tensors from open-source LLMs and run the same policy on real activations. After that, the policy should be integrated into an inference runtime to measure perplexity, deterministic token agreement, TTFT, TPOT, tokens/s, p95/p99 latency, and memory footprint. The current block-importance proxy should be replaced or compared with attention-mass, calibration-loss, and layer-sensitive estimators. Finally, LAMP-KV should be compared against fixed KIVI-style quantization, KVQuant-style outlier-aware quantization, QAQ-style quality adaptation, and adaptive bit-allocation baselines.

9 Conclusion

LAMP-KV studies a practical policy question for memory-constrained LLM inference: how aggressively should the KV cache be quantized under changing memory pressure? The pilot results show that a simple block-level policy can smoothly transition from conservative INT8 behavior to aggressive INT4 behavior and produce useful intermediate memory/error tradeoffs. The work is not a replacement for established KV quantization methods; rather, it is a policy layer and reproducible artifact for studying adaptive precision selection. The accompanying scripts and README are intended for repository-based peer review and future extension to real KV tensors and end-to-end LLM evaluation. The project repository is available at https://github.com/johncheungmk/LAMP-KV.

References

[1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, 2023.

[2] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache," in *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[3] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization," in *Advances in Neural Information Processing Systems*, 2024.

[4] S. Dong, W. Cheng, J. Qin, and W. Wang, "QAQ: Quality Adaptive Quantization for LLM KV Cache," arXiv preprint arXiv:2403.04643, 2024.

[5] Z. Zhang et al., "H_2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models," in *Advances in Neural Information Processing Systems*, 2023.

[6] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, "SnapKV: LLM Knows What You are Looking for Before Generation," in *Advances in Neural Information Processing Systems*, 2024.

[7] S. P. H. Boroujeni, N. Mehrabi, P. Woods, G. Hillesheim, and A. Razi, "Don't Waste Bits! Adaptive KV-Cache Quantization for Lightweight On-Device LLMs," arXiv preprint arXiv:2604.04722, 2026.

[8] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Advances in Neural Information Processing Systems*, 2022.

[9] Y. Sheng et al., "FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU," in *Proceedings of Machine Learning and Systems*, 2023.

[10] J. Cheung, "LAMP-KV supplementary artifact and testing scripts," GitHub repository, 2026. Available: https://github.com/johncheungmk/LAMP-KV.